

Reflektion över arkitekturval och säkerhet i två AWS-infrastrukturer

I det här projektet arbetade jag med två olika infrastrukturer i AWS. Den första infrastrukturen bestod av ett Spring Boot API på EC2 som kommunicerade med en MySQL-databas i RDS och exponerades via Lambda och API Gateway. Den andra infrastrukturen handlade om uppladdning och nedladdning av bilder med hjälp av Lambda och S3.

Syftet med projektet var att förstå hur AWS-tjänster samarbetar och hur resurser skyddas med hjälp av IAM-roller, Security Groups och privat eller publik åtkomst.

Arkitektur 1 – EC2, RDS, Lambda och API Gateway

I den första delen av projektet byggdes en arkitektur där användaren skickar anrop till API Gateway, som sedan skickar vidare trafiken till en Lambda-funktion. Lambda fungerar som en proxy och kommunicerar med ett Spring Boot API som körs på en EC2-instans. API:et kommunicerar sedan med en MySQL-databas i RDS.

EC2 användes i projektet för att köra Spring Boot-applikationen och gav möjlighet att förstå hur en backend-applikation driftsätts och hanteras i AWS. RDS användes som databastjänst eftersom AWS hanterar exempelvis backup, uppdateringar och stabilitet automatiskt.

Lambda-funktionen användes som en mellanhand mellan API Gateway och EC2. Det gjorde att både EC2 och RDS kunde ligga i privata subnät utan att vara öppna mot internet. API Gateway blev därför den enda publika ingången till systemet.

Det här är en säker arkitektur eftersom:

- API Gateway är den enda publika tjänsten
- Lambda tar emot och vidarebefordrar trafiken
- EC2 och RDS ligger skyddade i privata nätverk
- databasen aldrig exponeras direkt mot internet

Flödet fungerade på följande sätt:

1. Klienten skickar ett anrop till API Gateway.
2. API Gateway skickar vidare anropet till Lambda.
3. Lambda skickar vidare anropet till EC2-instansen.
4. Spring Boot-applikationen kommunicerar med databasen i RDS.
5. Svaret skickas tillbaka samma väg.

Detta gjorde att användaren aldrig kommunicerade direkt med backend eller databasen, vilket ökade säkerheten.

Arkitektur 2 – S3 och Lambda för bildhantering

I den andra delen av projektet byggdes en lösning för uppladdning och nedladdning av bilder med hjälp av S3 och Lambda.

I projektet användes:

- S3 för lagring av bilder

- Lambda för logik
- API Gateway för att ta emot anrop

Det är en enkel och skalbar lösning som passar bra för filhantering.

Upload fungerade på följande sätt:

1. Klienten skickar en bild till API Gateway.
2. Lambda tar emot bilden och använder PutObject för att spara den i S3.
3. Lambda returnerar statuskod och filens key.

Download fungerade så här:

1. Klienten skickar en key till API Gateway.
2. Lambda hämtar filen från S3 med GetObject.
3. Lambda returnerar filen eller en presigned URL.

Det här flödet fungerade både när bucketen var privat och när den var publik.

IAM-roller

IAM-roller fungerar ungefär som behörighetskort. Varje AWS-tjänst får bara göra det den behöver och inget mer.

Lambda-funktionen hade en IAM-roll som bland annat gav rättigheter att:

- skriva loggar till CloudWatch
- köras i VPC genom AWSLambdaVPCLambdaAccessExecutionRole

Detta behövdes för att Lambda skulle kunna nå EC2-instansen i det privata nätverket.

EC2-instansen hade inga extra rättigheter eftersom den inte behövde kommunicera med andra AWS-tjänster. Inga hårdkodade access keys användes heller i koden, vilket är viktigt för säkerheten.

IAM-roller var en av de mest utmanande delarna i projektet. Det tog tid att förstå hur IAM-policys fungerar och varför breda rättigheter som s3:* är osäkra. Jag behövde lära mig hur man begränsar rättigheter till en specifik bucket eller resurs.

I S3-delen fick Lambda-funktionen endast rättigheter för:

- s3:PutObject
- s3:GetObject

Och bara för den specifika bucketen. Det betyder att Lambda inte kunde läsa andra buckets eller få admin-rättigheter. Detta följer principen "least privilege", alltså minsta möjliga behörighet.

Security Groups

Security Groups fungerar som brandväggar som styr vilken trafik som får komma in och ut.

EC2-instansen hade regler som gjorde att:

- port 8080 bara var öppen för Lambda-funktionen

- port 22 bara var öppen för min egen IP-adress

RDS-databasen hade regler som gjorde att:

- port 3306 bara var öppen för EC2-instansen

Det betyder att:

- ingen kan nå databasen direkt från internet
- ingen kan nå backend direkt
- all trafik måste gå via API Gateway och Lambda

Security Groups var också en stor utmaning. Jag behövde förstå hur trafiken styrs mellan Lambda, EC2 och RDS och hur man öppnar rätt portar utan att öppna för mycket.

Det var först när jag ritade upp trafikflödet som jag verkligen började förstå hur reglerna fungerade tillsammans och varför varje regel behövdes.

Privat och publik åtkomst

En av de största utmaningarna under projektet var att förstå skillnaden mellan publika och privata resurser i AWS.

Jag blandade ofta ihop vad som var publikt och vad som bara var åtkomligt internt. API Gateway var publik eftersom användare måste kunna nå API:et via internet. Lambda, EC2 och RDS låg däremot i privata nätverk och kunde inte nås direkt utifrån.

Det tog tid att förstå hur trafiken faktiskt flödade genom infrastrukturen:

1. användaren når API Gateway
2. API Gateway kommunicerar med Lambda
3. Lambda kommunicerar internt med EC2
4. EC2 kommunicerar internt med RDS

När jag väl förstod detta blev det mycket tydligare hur AWS-tjänster samarbetar och varför privata resurser är viktiga för säkerheten.

I S3-delen testades både privata och publika buckets. När bucketen var privat kunde filer inte öppnas direkt via URL utan man behövde använda Lambda eller presigned URL. När bucketen var publik kunde alla öppna filerna direkt via URL, vilket gjorde lösningen mindre säker.

Jag hade också svårt att förstå skillnaden mellan presigned URL och direkt GetObject. Först trodde jag att presigned URLs alltid behövdes, men senare förstod jag att Lambda kan returnera filer direkt från en privat bucket om den har rätt IAM-behörigheter.

En presigned URL fungerar som en tillfällig nyckel till en specifik fil. Den fungerar bara under en begränsad tid och är säkrare än att göra hela bucketen publik.

Problem och lärdomar

När Lambda placerades i VPC uppstod flera timeout-problem. Problemet berodde på att Lambda inte hade rätt Security Group-regler för att nå EC2-instansen.

Det tog tid att förstå hur Lambda, subnät och Security Groups måste konfigureras tillsammans för att kommunikationen ska fungera. Detta lärde mig hur viktigt nätverk och säkerhetsregler är i AWS.

En annan svårighet var att förstå hur S3-keys fungerar. Först trodde jag att keyn behövde skrivas manuellt, men senare förstod jag att Lambda kan skapa unika keys automatiskt, exempelvis genom timestamps.

Den största utmaningen i projektet var att förstå helheten. AWS består av många olika tjänster och det tog tid att förstå hur API Gateway, Lambda, EC2, RDS och S3 samarbetar genom nätverk, säkerhet och trafikflöden.

Slutsats

Projektet gav mig en mycket bättre förståelse för hur man bygger säkra och skalbara lösningar i AWS.

Jag lärde mig:

- hur IAM-roller begränsar vad tjänster får göra
- hur Security Groups fungerar som brandväggar
- hur privat och publik åtkomst påverkar säkerheten
- hur API Gateway fungerar som en kontrollerad ingång
- hur viktigt det är att inte öppna portar i onödan

När jag väl förstod trafikflödet mellan tjänsterna blev hela infrastrukturen mycket tydligare. Projektet visade också hur viktigt det är att tänka på säkerhet redan från början när man designar en molninfrastruktur.